

# Step 1 — A Script That Sends One Message and Prints the Reply — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

**Story — Step 1:** before any chat, memory or agents, prove the three foundations work together: settings load from `.env`, logging works, and one call to the model comes back. Each piece goes in its final folder from day one, so later steps only *add* files. **If not:** a wrong key or a broken import would first show up in Step 3, mixed in with ticket logic, and you'd debug three things at once.

After this step the project already runs: `python -m supportdesk.main` sends one message and prints the reply.

## The Doc01 files (the same in both versions)

These three files are Doc01's Build Task code (Intermediate, Approach 1), **unchanged** — only the file locations and the `from supportdesk...` import line differ.

**Story — `src/supportdesk/exceptions.py`:** `load_config()` needs a way to say "a required setting is missing" that's more specific than a bare `Exception`. **If not:** `main.py` would have to catch every `Exception`, which also hides real bugs.

```
# src/supportdesk/exceptions.py
# why: gives load_config() its own error type, so main.py can catch a
# missing setting specifically, instead of catching every Exception.
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass
```

**Story — `src/supportdesk/config.py`:** the one place the project reads its settings from, failing loudly at startup. **If not:** every file would read `os.environ` itself, and a missing key would surface as a confusing `AuthenticationError` later.

```
# src/supportdesk/config.py
# how: os.getenv() is how we read env vars
import os
# why: @dataclass auto-builds __init__ for a typed holder
from dataclasses import dataclass
```

```

# how: reads .env and loads it into os.environ
from dotenv import load_dotenv
# why: a specific error type, not a bare Exception
from supportdesk.exceptions import MissingConfigError

@dataclass
class Config:
    # why: a typed object, not a plain dict – your editor autocompletes
    # .openai_api_key and .log_level, and catches a typo'd field name.
    openai_api_key: str
    log_level: str

def require_env(key: str) -> str:
    # why: one place that turns a missing/empty env var into a clear,
    # named error, instead of repeating the same if-check for every key.
    # when: called once per required key, inside load_config().
    value = os.getenv(key)
    if value is None or value == "":
        # how: both "never set" (None) and "set but blank" (empty string)
        # count as missing – an empty API key is never usable either way.
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config() -> Config:
    # why: the one function every other file calls to get settings – so a
    # missing key fails loudly here, at startup, not confusingly later.
    # when: call this once, at the top of main(), before anything else.
    # how: reads .env into os.environ; must run before os.getenv() below
    load_dotenv()
    # how: required – raises if missing
    api_key = require_env("OPENAI_API_KEY")
    # how: optional – "INFO" if unset
    log_level = os.getenv("LOG_LEVEL", "INFO")
    return Config(openai_api_key=api_key, log_level=log_level)

```

`load_dotenv()` looks for `.env` starting from this file's folder and walking up, so it finds the `.env` in the repo root.

**Story — `src/supportdesk/utils/logger.py`** : every file logs the same way, from one setup function. **If not:** each file would configure its own handler, and some lines would print twice.

```

# src/supportdesk/utils/logger.py
import logging
import os

def get_logger(name: str) -> logging.Logger:
    # why: gives every file in the project the same logging setup from one
    # place, instead of each file configuring handlers its own way.
    # when: logging.getLogger(name) always returns the same object for the
    # same name, so calling this again in the same run is safe and cheap.
    logger = logging.getLogger(name)

    if not logger.handlers:
        # how: only attach a handler the first time this name is
        # configured – without this guard, every line would print twice.
        level_name = os.getenv("LOG_LEVEL", "INFO")
        # how: falls back to INFO if the env value is invalid
        level = getattr(logging, level_name.upper(), logging.INFO)

        # how: prints to the console
        handler = logging.StreamHandler()
        handler.setLevel(level)
        # why: every line carries a timestamp/source/level, not just the
        # bare message
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        formatter = logging.Formatter(log_format)
        handler.setFormatter(formatter)

        logger.addHandler(handler)
        # how: the logger's own level must be at least this low, or the
        # handler never even sees the message
        logger.setLevel(level)

    return logger

```

Also create the empty `src/supportdesk/models/__init__.py` and `src/supportdesk/utils/__init__.py`, so both folders are importable packages.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Basic Version

### Approach 1 — the direct way

**Story — `models/llm.py` (Basic):** two small functions — build the client, send one message. **If not:** `main.py` would call the SDK directly, and every later file would copy that code.

```
# src/supportdesk/models/llm.py
from openai import OpenAI

def create_client(config):
    return OpenAI(api_key=config.openai_api_key)

def send_message(client, history, user_input):
    history.append({"role": "user", "content": user_input})
    response = client.chat.completions.create(
        model="gpt-4o-mini", messages=history
    )
    reply = response.choices[0].message.content
    history.append({"role": "assistant", "content": reply})
    return reply
```

**Story — `main.py` (Basic):** load settings, build the client, send one message. **If not:** there'd be nothing to run yet.

```
# src/supportdesk/main.py
from supportdesk.config import load_config
from supportdesk.models.llm import create_client, send_message

def main():
    config = load_config()
    client = create_client(config)
    history = []
    reply = send_message(client, history, "What's 2+2?")
    print(reply)
```

```
if __name__ == "__main__":  
    main()
```

**Run it** (from the repo root): `python -m supportdesk.main`

**Expected output:**

```
2 + 2 equals 4.
```

This works. It has no type hints, no logging, and a missing key shows as a raw `MissingConfigError` traceback. A wrong key isn't found until `send_message()` fails — mixed in with the real test.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

### Approach 1 — typed, logged, and the key checked at startup

**Story** — `models/llm.py`: the only file in the project that talks to the OpenAI SDK. It adds a `MODEL` constant and a startup key check. The key check sends a 1-token request, which costs almost nothing and tests the exact endpoint every agent will use. **If not:** a wrong key would first appear as a crash in the middle of the user's first real message.

```
# src/supportdesk/models/llm.py
import openai
from openai import OpenAI

from supportdesk.config import Config

# why: one place to change the model for every agent in the project
MODEL = "gpt-4o-mini"

def create_client(config: Config) -> OpenAI:
    # why: the key comes from the one validated place (load_config()),
    # not from the SDK quietly reading the environment by itself.
    # how: the client already retries rate limits, timeouts and 5xx
    # errors 2 times with backoff (Doc04) – so no retry loop of our own.
    return OpenAI(api_key=config.openai_api_key)

def check_api_key_at_startup(client: OpenAI) -> bool:
    # why: a bad key should fail once, clearly, before the user types
    # anything – not as a crash in the middle of their first message.
    # how: a 1-token reply costs almost nothing, and it tests the exact
    # endpoint every agent in this app depends on.
    try:
        client.chat.completions.create(
            model=MODEL,
            messages=[{"role": "user", "content": "ping"}],
            max_tokens=1,
        )
    except openai.AuthenticationError:
        return False
    return True
```

```
def send_message(client: OpenAI, history: list[dict], user_input: str) -> str:
    # why: Doc04's function, unchanged except for MODEL – history is the
    # memory, so the user turn and the reply are both appended to it.
    history.append({"role": "user", "content": user_input})
    response = client.chat.completions.create(model=MODEL, messages=history)
    reply = response.choices[0].message.content
    history.append({"role": "assistant", "content": reply})
    return reply
```

**Story — `main.py`** : the entry point. Settings first, then the logger, then the key check, then the one real call. Each failure gets one clear line and a clean exit. **If not:** a missing or wrong key would print a long traceback that means nothing to a user.

```
# src/supportdesk/main.py
import sys

from supportdesk.config import load_config
from supportdesk.exceptions import MissingConfigError
from supportdesk.models.llm import (
    check_api_key_at_startup,
    create_client,
    send_message,
)
from supportdesk.utils.logger import get_logger

def main() -> None:
    # why: config first – a missing OPENAI_API_KEY stops the app here,
    # with one clear line, before anything else runs
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    # when: after load_config(), so LOG_LEVEL from .env is already loaded
    logger = get_logger(__name__)
    client = create_client(config)
    if not check_api_key_at_startup(client):
        logger.error("Authentication failed – check OPENAI_API_KEY.")
        print("Login failed – check OPENAI_API_KEY in your .env file.")
        sys.exit(1)

    logger.info("Sending a test message to the model")
```

```

history: list[dict] = []
reply = send_message(client, history, "What's 2+2?")
print(reply)

if __name__ == "__main__":
    main()

```

**Run it** (from the repo root): `python -m supportdesk.main` — or just `supportdesk` .

**Expected output** (your timestamp and the model's wording will differ):

```

2026-09-25 10:00:00,123 __main__ INFO Sending a test message to the model
2 + 2 equals 4.

```

The logger names itself `__main__` here, because `python -m` runs `main.py` under that name.

### Check the failure paths:

- Remove `OPENAI_API_KEY` from `.env` → `Setup problem: Required environment variable is missing: OPENAI_API_KEY`
- Set it to `sk-fake-key-123` → an `ERROR` log line, then `Login failed – check OPENAI_API_KEY in your .env file.`

Both exit with code 1 and no traceback.

**Difference from Basic:** full type hints; a `MODEL` constant instead of the model name typed in each call; a logger; and two clean exits — a missing key is caught at `load_config()` , a wrong key at `check_api_key_at_startup()` , both before the real call.

**Which one should you actually write?** Intermediate. Every later step keeps `check_api_key_at_startup()` and this `main()` shape, and Step 3's error handling builds straight on top of it.